DMRG^{++}: A Pedagogical Companion

Density Matrix Renormalization Group, from first principles to working code

Documentation for the `dmrg_pp` library

April 23, 2026

Abstract

This document is the theoretical and algorithmic companion to the `dmrg_pp` library. It develops everything needed to understand the two-site finite-system Density Matrix Renormalization Group (DMRG) algorithm from scratch — starting with the curse of dimensionality of quantum many-body problems and ending with a fully derived effective-Hamiltonian update — and maps every concept to a concrete location in the source tree. The intended audience is a graduate student or researcher who has seen quantum mechanics and linear algebra but has never worked with tensor networks before.

Contents

1 Why DMRG? 2

2 Tensor-network notation in 90 seconds 2

3 Matrix Product States 3
- 3.1 Definition . . . 3
- 3.2 Why this is a good idea . . . 3
- 3.3 Canonical forms . . . 3

4 Truncated SVD: the workhorse 4

5 Matrix Product Operators 5
- 5.1 The finite-state-machine MPO builder . . . 5
- 5.2 Fermions and the Jordan–Wigner string . . . 6

6 The two-site DMRG algorithm 6
- 6.1 Environments . . . 6
- 6.2 The local effective Hamiltonian . . . 7
- 6.3 Lanczos eigensolver . . . 8
- 6.4 Update + truncation . . . 8
- 6.5 Pseudocode . . . 9

7 Measurements 9

8 Worked examples (reproducible) 9
- 8.1 Heisenberg antiferromagnet, $L = 32$ . . . 10
- 8.2 Critical TFIM and the central charge . . . 10
- 8.3 Hubbard chain at half filling . . . 10

9 Convergence diagnostics and good practice 10

10 Mapping document ↔ source 11

Further reading 11

1 Why DMRG?

A quantum lattice model on L sites with local Hilbert space dimension d lives in

$$\mathcal{H}_L = \bigotimes_{i=0}^{L-1} \mathbb{C}^d, \qquad \dim \mathcal{H}_L = d^L. \tag{1}$$

For a spin-$\frac{1}{2}$ chain ($d = 2$) on $L = 50$ sites this is already $\approx 1.13 \times 10^{15}$ complex amplitudes — larger than the memory of any conceivable single computer. Exact diagonalization is therefore limited to $L \lesssim 30$ in practice.

DMRG escapes this barrier by exploiting a remarkable empirical fact: the ground states of *gapped, local* 1D Hamiltonians have only *area-law* entanglement[1]. An area-law state on a 1D chain can be compressed into a *Matrix Product State* (MPS) of bounded bond dimension, with cost *linear* in L. DMRG is the variational algorithm that finds the optimal MPS approximation to the ground state.

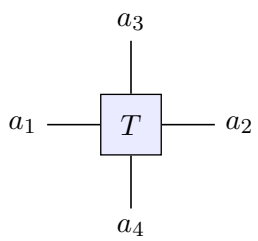
Code orientation. The library top level (`src/dmrg_pp/__init__.py`) re-exports the five user-facing names that drive most calculations:

```
from dmrg_pp import MPS, MPO, MPOBuilder, DMRGConfig, run_dmrg
```

We will derive what each of these represents and how they fit together.

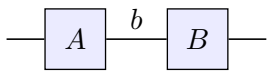
2 Tensor-network notation in 90 seconds

A rank-n tensor is, for our purposes, just an n-index numerical array $T_{a_1 a_2 \cdots a_n}$. We draw it as a node with n legs:



Connecting two legs means summing over their shared index — i.e. a contraction:

$$C_{ac} = \sum_b A_{ab}\, B_{bc} \quad \Longleftrightarrow \quad$$



. (2)

This is the entire graphical calculus. In the code, contractions are `numpy.tensordot` calls; the explicit axis lists in our routines are exactly the legs being joined in the diagram.

[1] Hastings (2007) made this rigorous: the ground state of a gapped, local 1D Hamiltonian satisfies $S(\rho_A) \le c$ for some constant c independent of $|A|$. Critical (gapless) systems satisfy a logarithmic correction $S \sim \frac{c}{6} \log L$ from CFT.

3 Matrix Product States

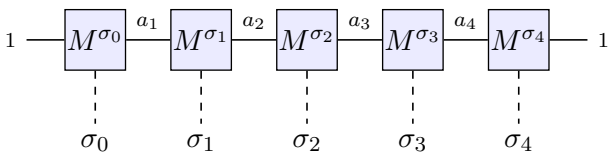
3.1 Definition

A wave-function $|\psi\rangle \in \mathcal{H}_L$ has d^L amplitudes $\psi_{\sigma_0\sigma_1\cdots\sigma_{L-1}}$. A *Matrix Product State* factorises these amplitudes as a chain of matrix products,

$$\boxed{\psi_{\sigma_0\sigma_1\cdots\sigma_{L-1}} \;=\; \sum_{a_1,\ldots,a_{L-1}} M^{\sigma_0}_{1,\,a_1}\, M^{\sigma_1}_{a_1,\,a_2}\,\cdots M^{\sigma_{L-1}}_{a_{L-1},\,1}} \tag{3}$$

where for each site i and each physical configuration σ_i, M^{σ_i} is a $D_i \times D_{i+1}$ matrix. The *virtual* (or *bond*) dimensions D_i control the amount of entanglement the state can carry; the boundary bonds $D_0 = D_L = 1$ make the outer brackets reduce to scalars.

Diagrammatically:



Each tensor M^{σ_i} corresponds to a single rank-3 NumPy array of shape $(D_i,\, d,\, D_{i+1})$, with the convention:

$$\text{axis } 0 = \text{left virtual bond}, \quad \text{axis } 1 = \text{physical leg}, \quad \text{axis } 2 = \text{right virtual bond}.$$

This is exactly the layout enforced by the constructor of `src/dmrg_pp/states/mps.py`:

```
def __init__(self, tensors, *, centre=None, copy=True):
    ...
    if t.ndim != 3:
        raise ValueError(f"site {i}: tensor must be rank-3, got ndim={t.ndim}")
    if t.shape[0] != prev_right:
        raise ValueError(f"site {i}: left bond {t.shape[0]} != "
                         f"previous right bond {prev_right}")
```

3.2 Why this is a good idea

The total parameter count of an MPS with bounded bond dimension $D_i \le \chi$ is

$$\#\text{params} \;\le\; L\, d\, \chi^2, \tag{4}$$

which is *linear* in L. This is exponentially smaller than d^L the moment χ is bounded. The non-trivial physics statement is that for gapped local 1D Hamiltonians, χ *can* be bounded (independent of L, modulo polylog corrections at criticality) while still capturing the ground-state physics to any desired accuracy.

3.3 Canonical forms

The MPS representation is *not unique*: inserting any invertible matrix X on a bond and its inverse X^{-1} on the next site gives a different representation of the same wave-function. We exploit this gauge freedom to put the MPS into a convenient *canonical form*.

Definition 3.1 (Left- and right-canonical sites)**.** A site tensor $A^{\sigma}_{a,b}$ is **left-canonical** if, viewed as a $(Dd) \times D'$ matrix, it is a left-isometry:

$$\sum_{a,\sigma} (A^{\sigma}_{a,b})^* \, A^{\sigma}_{a,b'} \;=\; \delta_{b\,b'}. \tag{5}$$

Right-canonical sites satisfy the mirror condition $\sum_{\sigma,b} A^{\sigma}_{a,b} \, (A^{\sigma}_{a',b})^* = \delta_{a\,a'}$.

Definition 3.2 (Mixed canonical form)**.** An MPS is in **mixed canonical form with orthogonality centre** c if sites $i < c$ are left-canonical, sites $i > c$ are right-canonical, and the centre site carries the entire normalisation:

$$\|\psi\|^2 \;=\; \left\| M^{(c)} \right\|_F^2. \tag{6}$$

Moving the centre by one site costs a single **QR decomposition** — this is what the helpers `_left_canonicalize_site` and `_right_canonicalize_site` of the `MPS` class do in `src/dmrg_pp/states/mps.py`. The class tracks the current centre in `self._centre` so that subsequent operations (norms, local expectation values, ...) are $\mathcal{O}(1)$ rather than $\mathcal{O}(L)$.

Why we sign-fix the QR. The QR decomposition is unique only up to a diagonal unitary; standard LAPACK does not fix this freedom. Our implementation in `src/dmrg_pp/tensors/linalg.py` forces the diagonal of R to be real and non-negative, so that `left_qr`(A) is a deterministic function of A — crucial for reproducible tests.

4 Truncated SVD: the workhorse

The single most important fact in DMRG is the optimality of singular-value truncation.

Theorem 4.1 (Eckart–Young–Mirsky)**.** *Let $M \in \mathbb{C}^{m\times n}$ have SVD $M = U\Sigma V^\dagger$ with singular values $s_1 \geq s_2 \geq \cdots \geq s_{\min(m,n)} \geq 0$. Then for any χ, the matrix $M_\chi = U_{:\chi}\Sigma_\chi V^\dagger_{:\chi}$ obtained by keeping only the largest χ singular values is the unique*[2] *minimiser of $\|M - X\|_F$ over all rank-χ matrices X. The truncation error is*

$$\|M - M_\chi\|_F^2 \;=\; \sum_{i>\chi} s_i^2. \tag{7}$$

In MPS language, this means: if we want to compress a bond from dimension D down to χ, the SVD prescription is *provably optimal*.

The DMRG sweep engine of section 6 truncates after every two-site update. The library exposes the relevant primitive in `src/dmrg_pp/tensors/linalg.py`:

```
def truncated_svd(matrix, *, max_bond=None, cutoff=0.0,
                  min_bond=1, renormalize=False):
    """Truncated SVD with rich diagnostics.

    Decomposes matrix ~ U @ diag(s) @ Vh keeping at most max_bond
    singular values and discarding any with cumulative weight below
    cutoff * sum(s**2).
    """
```

The *relative* cutoff convention (relative to the total Frobenius weight $\sum_i s_i^2$) is the same one used by ITensor and TenPy. We return a `TruncationInfo` dataclass containing the kept singular spectrum, the discarded weight, the largest discarded value, and the resulting bond dimension — all of which the DMRG sweep engine logs and persists.

[2] Up to genuine spectral degeneracies in the singular values $s_\chi = s_{\chi+1}$.

5 Matrix Product Operators

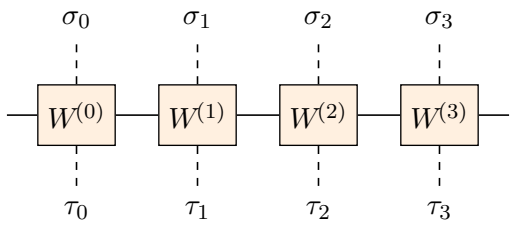
An MPO factorises an operator $\hat{H} : \mathcal{H}_L \to \mathcal{H}_L$ as

$$\hat{H} \;=\; \sum_{\{w\}} W^{\sigma_0\tau_0}_{1,\,w_1}\, W^{\sigma_1\tau_1}_{w_1,\,w_2} \cdots W^{\sigma_{L-1}\tau_{L-1}}_{w_{L-1},\,1}\; |\boldsymbol{\sigma}\rangle\,\langle\boldsymbol{\tau}|\,. \tag{8}$$

Each tensor W at site i is a rank-4 array of shape $(D_w,\, d,\, d,\, D'_w)$ with leg ordering

(left bond, σ output, τ input, right bond).

Diagrammatically:



The *action* convention is the natural one, $(\hat{H}\,|\psi\rangle)_{\boldsymbol{\sigma}} = \sum_{\boldsymbol{\tau}} H_{\boldsymbol{\sigma},\boldsymbol{\tau}}\psi_{\boldsymbol{\tau}}$, so σ is the *output* (ket) leg and τ the *input* (bra) leg. This is enforced in `src/dmrg_pp/operators/mpo.py`.

5.1 The finite-state-machine MPO builder

How do we obtain the local W tensors for a Hamiltonian like the Heisenberg chain

$$\hat{H}_{\mathrm{XXZ}} \;=\; \sum_{i=0}^{L-2} \Big[\, \tfrac{J_{xy}}{2}\big(\hat{S}^+_i\hat{S}^-_{i+1} + \hat{S}^-_i\hat{S}^+_{i+1}\big) + J_z\hat{S}^z_i\hat{S}^z_{i+1} \,\Big] \;+\; h\sum_i \hat{S}^z_i \;? \tag{9}$$

The cleanest construction views the MPO as a finite-state automaton on the virtual bond. The bond at position b carries three classes of states:

(a) **Start**: "no Hamiltonian term has begun yet to my left."

(b) **Transit**$_\alpha$: "term α has begun but not yet finished" — one bond state per term currently in flight.

(c) **End**: "every term whose support is to my left has finished emitting."

The local W tensors are the transition matrices of this automaton, decorated with the local operators for the transitions that emit them. The "always identity" loops on **Start** and **End** ensure that terms whose support has not yet (or already) been visited contribute the identity locally.

Example 5.1 (Transverse-field Ising chain)**.** For $\hat{H} = -J\sum_i \sigma^z_i\sigma^z_{i+1} - g\sum_i \sigma^x_i$ the FSM has $D = 3$: {Start, Transit$_Z$, End}. The internal W tensors read, in the basis (Start, Transit$_Z$, End):

$$W \;=\; \begin{pmatrix} \hat{\mathbb{1}} & \sigma^z & -g\,\sigma^x \\ 0 & 0 & -J\,\sigma^z \\ 0 & 0 & \hat{\mathbb{1}} \end{pmatrix}, \tag{10}$$

which is exactly the well-known $D = 3$ Ising MPO.[3]

[3] Reading entries right-to-left: the lower-right $\hat{\mathbb{1}}$ block is "stay in End"; the upper-left $\hat{\mathbb{1}}$ is "stay in Start"; the column above End reads "end the term by emitting $-g\sigma^x$ (single-site) or $-J\sigma^z$ (closing a two-site term)".

This construction is implemented in `src/dmrg_pp/operators/builder.py`. The user-facing API is symbolic:

```
from dmrg_pp.operators.builder import MPOBuilder
from dmrg_pp.operators.local_ops import spin_half_ops

ops = spin_half_ops()
b = MPOBuilder(L=L, local_ops=ops)
for i in range(L - 1):
    b.add(0.5, (i, "Sp"), (i + 1, "Sm"))
    b.add(0.5, (i, "Sm"), (i + 1, "Sp"))
    b.add(1.0, (i, "Sz"), (i + 1, "Sz"))
mpo = b.build() # produces D=5 isotropic-Heisenberg MPO
```

For nearest-neighbour two-body Hamiltonians the construction yields the optimal bond dimension out of the box; for longer-range terms the bond dimension grows as the number of in-flight terms across each bond.

5.2 Fermions and the Jordan–Wigner string

For spinful fermions on a 1D chain the canonical anticommutation relations $\{c_{i\sigma}, c^\dagger_{j\sigma'}\} = \delta_{ij}\delta_{\sigma\sigma'}$ do not factor across sites. The Jordan–Wigner transformation

$$c_{i\sigma} \;=\; K_i\, \tilde{c}_{i\sigma}, \qquad K_i \;=\; \prod_{k<i} F_k, \qquad F_k \;=\; (-1)^{N_k}, \tag{11}$$

trades the anticommutation for a non-local string operator K_i acting on all sites to the left of i. For nearest-neighbour bilinears this collapses:

$$\boxed{c^\dagger_{i\sigma}\, c_{i+1\,\sigma} \;=\; (\tilde{c}^\dagger_{i\sigma}\, F_i) \otimes \tilde{c}_{i+1\,\sigma}.} \tag{12}$$

The F-string is absorbed into the *left* site, so the symbolic builder of section 5.1 sees a single local operator per site and produces a small-bond MPO. The Hubbard implementation in `src/dmrg_pp/models/hubbard.py` pre-computes the four decorated operators `Cup_dag_F`, `F_Cup`, `Cdn_dag_F`, `F_Cdn` for exactly this reason.

6 The two-site DMRG algorithm

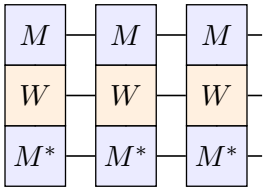
We are now ready to derive the central algorithm. The setup is:

- A Hamiltonian as an MPO $\hat{H}$ (section 5).
- A trial MPS $|\psi\rangle$ (section 3), in mixed canonical form with centre at site 0.
- A bond-dimension cap χ and an SVD cutoff ε.

We want to minimise $\langle\psi|\,\hat{H}\,|\psi\rangle \,/\, \langle\psi|\psi\rangle$ over the MPS manifold of bond dimension $\leq \chi$. DMRG does this by a sequence of *local* optimisations.

6.1 Environments

Define the **left environment** at bond b as the contraction of all sites $0, \ldots, b-1$:



L_b has three open legs $\to$ shape (D, D_w, D)

Symbolically:

$$L_b^{a'\,w\,a} \;=\; \sum_{\substack{\boldsymbol{\sigma}_{<b},\,\boldsymbol{\tau}_{<b}\\ a_{1\dots b-1},\,a'_{1\dots b-1}}} \left(M^{\sigma_0\,*}_{1,a'_1}\cdots M^{\sigma_{b-1}\,*}_{a'_{b-1},a'}\right)\left(W^{\sigma_0\tau_0}_{1,w_1}\cdots W^{\sigma_{b-1}\tau_{b-1}}_{w_{b-1},w}\right)\left(M^{\tau_0}_{1,a_1}\cdots M^{\tau_{b-1}}_{a_{b-1},a}\right). \tag{13}$$

The **right environment** R_b is the mirror image, contracting sites $b, \dots, L-1$ from the right. The full energy is then any-bond

$$E \;=\; \langle\psi|\,\hat{H}\,|\psi\rangle \;=\; \sum_{a,w,a'} L_b^{a'wa}\,(\,\text{remainder at bond } b\,). \tag{14}$$

The library stores **both** environments in `src/dmrg_pp/algorithms/environments.py`, with the convention

$$\begin{aligned} L_{\text{envs}}[k] \;&=\; \text{contraction of sites } 0,\dots,k-1\text{, exposed at the left bond of site } k,\\ R_{\text{envs}}[k] \;&=\; \text{contraction of sites } k,\dots,L-1\text{, exposed at the same bond.}\end{aligned}$$

This dual-indexing has a beautiful corollary: the two-site update at bond $(i, i+1)$ needs precisely $L_{\text{envs}}[i]$, $R_{\text{envs}}[i+2]$, $W^{(i)}$, and $W^{(i+1)}$. Sweeping the centre by one site requires only a single environment refresh.

6.2 The local effective Hamiltonian

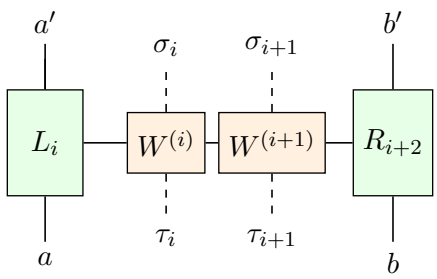
At bond $(i, i+1)$, contract the two adjacent MPS sites into a rank-4 *two-site block*:

$$\Theta^{\sigma_i\,\sigma_{i+1}}_{a,b} \;=\; \sum_c M^{\sigma_i}_{a,c}\,M^{\sigma_{i+1}}_{c,b}. \tag{15}$$

The variational energy as a function of Θ is

$$E[\Theta] \;=\; \frac{\langle\Theta|\;\hat{H}^{(i)}_{\text{eff}}\;|\Theta\rangle}{\langle\Theta|\Theta\rangle}, \qquad \hat{H}^{(i)}_{\text{eff}} \;=\; L_{\text{envs}}[i]\;\diamond\;W^{(i)}\;\diamond\;W^{(i+1)}\;\diamond\;R_{\text{envs}}[i+2] \tag{16}$$

where "$\diamond$" denotes the appropriate tensor contraction shown below.



The new Θ_* is the lowest eigenvector of $\hat{H}_{\text{eff}}$, found by a Krylov method. We never form $\hat{H}_{\text{eff}}$ as a dense matrix (it would have size $(Dd^2D)^2$); we only need its **action** on a vector, which is the function `_apply_two_site_heff` in `src/dmrg_pp/algorithms/dmrg.py`:

```python
def _apply_two_site_heff(psi2_flat, L_env, W1, W2, R_env, shape):
    """Apply the two-site effective Hamiltonian to a flat state vector."""
    psi2 = psi2_flat.reshape(shape)
    tmp = np.tensordot(L_env, psi2, axes=([2], [0])) # (a',o,d1,d2,b)
    tmp = np.tensordot(tmp, W1, axes=([1, 2], [0, 2])) # (a',d2,b,d1',o')
    tmp = np.tensordot(tmp, W2, axes=([1, 4], [2, 0])) # (a',b,d1',d2',o'')
    tmp = np.tensordot(tmp, R_env, axes=([1, 4], [2, 1]))# (a',d1',d2',b')
    return tmp.reshape(-1)
```

The cost of one matvec is, with D_w the MPO bond,

$$\mathcal{O}(D^3\, d^2\, D_w \;+\; D^2\, d^2\, D_w^2). \tag{17}$$

Krylov methods like Lanczos converge in $\mathcal{O}(\log(1/\varepsilon))$ iterations on well-conditioned problems, giving the desired ground state extremely cheaply per local solve.

6.3 Lanczos eigensolver

The local solve is performed by `src/dmrg_pp/algorithms/eigensolver.py`, which wraps SciPy's ARPACK (`scipy.sparse.linalg.eigsh`) behind a uniform API and adds two fallbacks:

(1) For tiny effective Hilbert spaces (default threshold 32) it materialises $\hat{H}_{\text{eff}}$ and uses dense `eigh` — both faster and more accurate than ARPACK at this size.

(2) If ARPACK declines to converge it falls back to a hand-rolled re-orthogonalised Lanczos with 50–200 Krylov vectors.

The Lanczos recursion itself — starting from $v_0 = \Theta_{\text{init}}$ (i.e. *warm-starting* from the previous solution) — builds an orthonormal Krylov basis $\{v_0, v_1, \ldots, v_{k-1}\}$ via

$$\beta_j v_{j+1} \;=\; \hat{H}_{\text{eff}}\, v_j - \alpha_j v_j - \beta_{j-1} v_{j-1}, \qquad \alpha_j \;=\; \langle v_j,\, \hat{H}_{\text{eff}}\, v_j \rangle, \tag{18}$$

which produces a tridiagonal matrix $T = \text{tridiag}(\beta_{j-1}, \alpha_j, \beta_j)$ whose smallest eigenpair converges (Kaniel–Saad bounds) to the smallest eigenpair of $\hat{H}_{\text{eff}}$ exponentially fast in the spectral gap.

6.4 Update + truncation

Having solved $\hat{H}_{\text{eff}}\Theta_* = E_*\,\Theta_*$, we re-split the two-site tensor by SVD,

$$\Theta_{*\,a,b}^{\sigma_i\sigma_{i+1}} \xrightarrow[\text{reshape}]{} \Theta_*{}^{(a\sigma_i),(\sigma_{i+1}b)} \;=\; U^{(a\sigma_i),k}\, s_k\, V^{\dagger\, k,(\sigma_{i+1}b)}, \tag{19}$$

truncate to the largest χ singular values (or to the cutoff threshold, whichever is tighter), and put the result back as

$$\text{(right-going sweep)} \quad M_{a,k}^{\sigma_i} \;\leftarrow\; U^{(a\sigma_i),k}, \qquad M_{k,b}^{\sigma_{i+1}} \;\leftarrow\; (s_k V^\dagger)^{k,(\sigma_{i+1}b)}, \tag{20}$$

which leaves the orthogonality centre at site $i+1$, ready for the next local optimisation.

A complete *full sweep* consists of stepping through every bond in this fashion from $i = 0$ to $i = L-2$ (right-going), then back down to $i = 0$ (left-going). Convergence is monitored by the energy change per sweep and the spread of local energies within a sweep — a proxy for the Hellman–Feynman variance. See the `SweepStats` dataclass and the convergence checks in `src/dmrg_pp/algorithms/dmrg.py`.

6.5 Pseudocode

```
def two_site_dmrg(H, psi, chi, eps, n_sweeps):
    psi.orthogonalize(0) # left-canonicalize all sites
    L = build_left_envs(psi, H) # length (L+1) cache
    R = build_right_envs(psi, H)
    for sweep in range(n_sweeps):
        for direction in ['right', 'left']:
            for i in bond_order(direction):
                Theta = contract(psi[i], psi[i+1])
                E, Theta_star = lanczos_ground_state(
                    H_eff(L[i], H[i], H[i+1], R[i+2]),
                    initial=Theta)
                U, s, Vh, info = truncated_svd(
                    Theta_star.reshape(...),
                    max_bond=chi, cutoff=eps)
                if direction == 'right':
                    psi[i] = U
                    psi[i+1] = diag(s) @ Vh
                    refresh_left_env(L, i)
                else:
                    psi[i] = U @ diag(s)
                    psi[i+1] = Vh
                    refresh_right_env(R, i+1)
        if converged(E_history): break
    return psi, E
```

7 Measurements

Local observables. Once the centre sits on site i, the expectation value of any single-site operator $\hat{O}$ reduces to a $\mathcal{O}(D^2d^2)$ contraction:

$$\langle \hat{O}_i \rangle \;=\; \sum_{\sigma,\sigma',a,b} (M^{\sigma'}_{a,b})^* \, O_{\sigma'\sigma} \, M^{\sigma}_{a,b}, \tag{21}$$

implemented in `src/dmrg_pp/measurements/observables.py`. Sweeping the centre once across the chain produces the full vector $\{\langle \hat{O}_i \rangle\}_{i=0}^{L-1}$ in $\mathcal{O}(LD^3d^2)$ time.

Two-point correlators. For $\langle \hat{A}_i \hat{B}_j \rangle$ with $i < j$ we propagate a two-leg "transfer" environment of shape $(D_{\text{bra}}, D_{\text{ket}})$ from site i to site j. Optionally we insert a Jordan–Wigner string on every intermediate site (for fermionic correlators).

Entanglement. The Schmidt spectrum across any bond is read off after canonicalising through it:

$$|\psi\rangle \;=\; \sum_{\alpha=1}^{\chi} s_\alpha \left|\phi^L_\alpha\right\rangle \otimes \left|\phi^R_\alpha\right\rangle, \tag{22}$$

$$S(\rho_L) \;=\; -\sum_{\alpha} s_\alpha^2 \log s_\alpha^2. \tag{23}$$

The "entanglement Hamiltonian" spectrum is then $-2\log s_\alpha$. See `src/dmrg_pp/measurements/entanglement.py`.

8 Worked examples (reproducible)

The library ships three runnable scripts in `examples/` that exercise different parts of the codebase.

8.1 Heisenberg antiferromagnet, $L = 32$

```
from dmrg_pp import DMRGConfig, run_dmrg
from dmrg_pp.models import HeisenbergXXZ
model = HeisenbergXXZ(L=32, Jxy=1.0, Jz=1.0)
cfg = DMRGConfig(n_sweeps=8, max_bond=[16,32,64,64,96], cutoff=1e-10, seed=2026)
res = run_dmrg(model.mpo(), config=cfg)
```

Converges in $\sim 8\,\mathrm{s}$ to $E/L \approx -0.4374$, against the Bethe-ansatz infinite-chain limit $e_\infty = \frac{1}{4} - \ln 2 \approx -0.4431$ (the $\sim 1\%$ residual is the well-known $1/L^2$ open-boundary correction).

8.2 Critical TFIM and the central charge

The transverse-field Ising chain at $g = J$ is at the Ising quantum critical point, described by a $c = \frac{1}{2}$ conformal field theory. Cardy and Calabrese's formula predicts the bipartite entropy of an open chain to be

$$S(b) \;=\; \tfrac{c}{6}\,\log\!\Big[\tfrac{2L}{\pi}\sin\tfrac{\pi b}{L}\Big] + \text{const.} \tag{24}$$

The script `examples/tfim.py` runs DMRG, extracts the bond entropies via `bond_entropies`, and fits the slope of S vs. the "chord function" to recover c. At $L = 32$ a free-time run gives $c \approx 0.557$, within $\sim 11\%$ of the exact 0.5 — the rest is finite-size scaling.

8.3 Hubbard chain at half filling

`tests/test_dmrg.py` contains the integration test `test_dmrg_hubbard_small_matches_exact`, which runs DMRG on the $L = 4$, $U = 2$, $t = 1$ Hubbard chain and compares against full ED of the dense 256×256 Hamiltonian. The Jordan–Wigner machinery of `src/dmrg_pp/models/hubbard.py` brings the DMRG energy to within 10^{-7} of the exact ground state.

9 Convergence diagnostics and good practice

A successful DMRG run typically exhibits:

- Energy decreasing monotonically (to within machine precision).
- Per-sweep *energy variance* (the spread of local energies across bonds within one sweep) decaying to 0. This is a true Hellmann–Feynman check: it vanishes *only* when $|\psi\rangle$ is an exact eigenstate.
- Maximum truncation error decreasing as χ is ramped up.
- Mid-chain entanglement entropy converging to a χ-independent value.

A typical *failure* mode is converging to a low-lying excited state in a different symmetry sector — particularly common in the presence of an exact degeneracy (e.g. $\mathbb{Z}_2$ in the ferromagnetic Ising phase). Mitigations: adding a small symmetry-breaking field, increasing the random-seed diversity, or using single-site DMRG with subspace expansion (Hubig 2017) for the final sweeps.

BLAS thread thrashing. A surprisingly common — and surprisingly catastrophic — performance issue: for the small per-bond tensors that DMRG operates on ($D \lesssim 200$), the OpenBLAS/MKL thread-pool overhead dominates the actual flops. The library defaults to single-threaded BLAS via `threadpoolctl`; on a hyper-threaded WSL2 box this delivered a $\sim 100\times$ speed-up at $D = 64$. See `src/dmrg_pp/utils/threading.py`.

10 Mapping document ↔ source

Concept (this document)	Implementation
MPS, canonical forms (sections 3 and 3.3)	`src/dmrg_pp/states/mps.py`
Truncated SVD (section 4)	`src/dmrg_pp/tensors/linalg.py`, `truncated_svd`
MPO data structure (section 5)	`src/dmrg_pp/operators/mpo.py`
FSM-based MPO builder (section 5.1)	`src/dmrg_pp/operators/builder.py`
Local operator catalogs	`src/dmrg_pp/operators/local_ops.py`
Models (Heisenberg, TFIM, Hubbard)	`src/dmrg_pp/models/`
Environments	`src/dmrg_pp/algorithms/environments.py`
Effective Hamiltonian + matvec	`src/dmrg_pp/algorithms/dmrg.py`, `_apply_two_site_heff`
Lanczos eigensolver	`src/dmrg_pp/algorithms/eigensolver.py`
Two-site sweep driver (section 6)	`src/dmrg_pp/algorithms/dmrg.py`, `run_dmrg`
Local observables, correlators, entropies	`src/dmrg_pp/measurements/`
HDF5 persistence + YAML configs	`src/dmrg_pp/io/`
CLI (`dmrg run / info`)	`src/dmrg_pp/cli.py`
Integration tests vs. exact diagonalisation	`tests/test_dmrg.py`

Further reading

[1] S. R. White, *Density Matrix Formulation for Quantum Renormalization Groups*, Phys. Rev. Lett. **69**, 2863 (1992).

[2] U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, Annals of Physics **326**, 96 (2011).

[3] C. Hubig, I. P. McCulloch, U. Schollwöck, *Generic construction of efficient matrix product operators*, Phys. Rev. B **95**, 035129 (2017).

[4] J. Hauschild, F. Pollmann, *Efficient numerical simulations with Tensor Networks: Tensor Network Python (TeNPy)*, SciPost Phys. Lect. Notes **5** (2018).

[5] M. B. Hastings, *An area law for one-dimensional quantum systems*, J. Stat. Mech. **P08024** (2007).

[6] P. Calabrese, J. Cardy, *Entanglement entropy and quantum field theory*, J. Stat. Mech. **P06002** (2004).